



Technical Specifications

Project	Qwenta Menu Maker
----------------	-------------------

Version	Author	Date	Approval
1.0	Igor Vish	08/05/2026	Soufiane

The purpose of this document is to define and justify the technical specifications for Qwenta Menu Maker.

I. Technology Decisions

- Overview of the functional requirements and their technical solutions:

Requirement	Constraints	Solution	Description of the Solution	Supporting Arguments (2)
<i>Landing Page (not Logged In)</i>	<i>Whether user logged in or not this page is accessible to everyone</i>	<i>React, Sass for CSS</i>	<i>React is best for components for front end application</i>	<i>React and Sass are in the front of web development</i>
<i>Feature Improved for</i>	<i>Animations for Banner and geometric shapes of</i>	<i>CSS Animation for Banner</i>	<i>This is simple solution instead of using third party</i>	<i>Best for simple animations.</i>



<i>Landing Page (not Logged In)</i>	<i>sections</i>	<i>image, CSS animations to geometric shapes of sections</i>	<i>library</i>	
<i>Login Modal</i>	<i>Modal that opens when user logged in</i>	<i>react-modal</i>	<i>This React library simplifies the creation of powerful, accessible modal windows with minimal coding</i>	<i>We decided to develop in React. The library is consistence with this choice. This is the most commonly used library.</i>
<i>Sending authentication email from modal</i>	<i>User should enter email that should be authenticated and after authentication user is able to login to menu creation page</i>	<i>1. React, fetch api to backend to send email 2. 'resend' to send email 3. 'crypto' to generate token 4. mongoose to save token in database</i>	<i>Front end will handle states email, pending and success to move through modal screens Backend will send POST request, generate token , send email and saves token in database backend polls every 2 second for verification token from user's email</i>	<i>This is secure way to make sure user's email is not tempered</i>
<i>Creation bear token when user logged in</i>	<i>Verification user is logged in</i>	<i>Create bearer token using JavaScript</i>	<i>Create bearer token when user is logged and save it in local front end storage</i>	<i>Standard way to verify user is logged in</i>



<i>Dish category save, update</i>	<i>Modal to choose dish category</i>	<i>React-modal</i>	<i>Simplifies creation of modal window in React</i>	<i>We've decided to develop in React. The library is consistent with this choice. 2) This is the most commonly used.</i>
<i>Dish creation</i>	Restaurateurs need the ability to add a dish/dishes to their menu	<i>Fetch call in React, Rest call in backend</i>	<i>CRUD call for dish creation</i>	<i>Standard for saving, updating and deleting data in the front end.</i>
<i>Menu style</i>	Logged-in restaurateurs need the ability to customize the menu style	<i>Fetch call in React, Rest call to backend</i>	<i>CRUD call for dish.</i>	<i>Standard for saving, updating and deleting data in the front end.</i>
<i>PDF export</i>	It still technically mounts to the real DOM for a split second behind the scenes so the canvas engine can see it, but React manages the entire lifecycle	<i>react-to-pdf library. Rest call to save pdf in the backend</i>	<i>External library to convert menu that is in-memory react html to pdf.</i>	<i>Library dedicated to react.</i>
<i>Order a printed menu</i>	When user clicks to order a printed menu, it has to connect to Deliveroo application/website	<i>Fetch api in React Rest call in Backend to Deliveroo</i>	<i>Fetch api front end, Rest call to open Deliveroo website or application sending menu to be printed.</i>	<i>Since Deliveroo, rest call is simplest way to send menu to be printed and open Deliveroo web page in separate tab</i>



<i>Previous Menu</i>	Restaurateurs need the ability to view previously created menus	<i>Fetch api in React, Rest call backend</i>	Rest call to get menu, update menu, create new menu Menu should also have a date	<i>Standard for saving, updating and deleting data in the front end.</i>
<i>Legal Notice</i>	Users must be able to access the “Legal Notice” content in a modal window, and the “All Rights Reserved” information must be displayed.	<i>react-modal</i>	Modal for Legal Notice and All Right Reserved	<i>Library to ease use of modal in react since we are using react for Qwenta.</i>
<i>Prices</i>	Users need access to the Menu Maker prices	<i>window.open page in new tab</i>	New tab will have predefined prices page	<i>Easiest solution to display static page.</i>
<i>Share on instagram</i>	Restaurateurs need the ability to easily share their menu on Instagram	<i>Fetch api front end, Rest call backend</i>	Rest call to upload image and captions to Instagram account	<i>Standard for saving, updating and deleting data in the front end.</i>
<i>User information</i>	Restaurateurs need the ability to edit their user information	<i>Fetch api front end, Rest call backend</i>	Rest call to update user information Rest call to link multiple emails to user	<i>Standard for saving, updating and deleting data in the front end.</i>
<i>Dashboard</i>	Restaurateurs need the ability to access a	<i>Clickable cards in</i>	Standard javascript/react technique	<i>Easiest solution.</i>



	dashboard that includes Creating the menu, Distributing the men, Printing the menu, The last three Qwenta blog posts about Menu Maker.	<i>React/javascript</i>		
<i>Menu maker blog</i>	<i>Restaurateur needs the ability to create and post a blog</i>	<i>Rest call to save block and get it</i>	CRUD call to save blog	<i>Standard for saving, updating and deleting data in the front end</i>
<i>Restaurant branding</i>	Restaurateurs need the ability to create their restaurant's branding	<i>Rest call</i>	Create logo: Rest call to create /update/ delete Logo	<i>Standard for saving, updating and deleting data in the front end.</i>

II. Links With the Backend

- Which server language? *Node.js*
- Is an API needed: Yes
- Selected database: *MongoDb*



API Specifications

	Access Point	Description
POST	/api/auth/status	user status: EMAIL, PENDING, SUCCESS
POST	/api/auth/send-email	Send email to backend
POST	/api/menucategories	Create menues categories
GET	/api/menucategories/:id	Get menu category
PUT	/api/menucategories/:id	Update menu category
POST	/api/dishes	Create dish
GET	/api/dishes/:id	Get dish
PUT	/api/dishes/id	Update dish
POST	/api/menues	Create menu
GET	/api/menues/:id	Get menu
PUT	/api/menues/:id	Update Menu
DELETE	/api/menues/:id	Delete Menu
POST	/api/menuespdf	Save Menu as PDF
GET	/api/menuespdf/:id	Get PDF Menu
PUT	/api/menuespdf/:id	Update PDF Menu
POST	/api/instagrammenues	Create Instagram Square Menu



Models

```
User {  
  defaultEmail: String  
  additionalEmails [  
    { email: String }  
  ]  
  Payment {  
    creditCard: String  
    expirationDate: String  
    cvv: String  
  }  
}
```

```
Dish {  
  name: String  
  imageUrl: String  
  price: floating point number  
  description: String  
}
```

```
DishCategory {  
  name: String
```



```
}
```

```
Menu {  
    restaurantName: String  
    pdfUrl: String  
    squareImageUrl: String  
    color: String  
    font: String  
    dishCategories [  
        { dishCategory: DishCategory }  
    ]  
    Dishes [  
        { dish: Dish }  
    ]  
}
```

III. Domain and Hosting Provider Recommendations

—

- Domain name: qwentamenumaker.com
- Hosting provider name: IONOS
- Email address webgencia.com



IV. Accessibility

—

- Browser compatibility: Chrome, Firefox, Safari
- Device types: desktop, laptops, cell phone major types

V. Third-Party Services

- None

VI. Security Recommendations

—

- Bearer token for authentications
- Email verification through front end api and backend handling token and rest call to client's email
- All data are saved in backend

VII. Website Maintenance and Future Updates

- Maintenance for agreed price